Instituto Superior de Engenharia do Porto

Mestrado em Engenharia de Inteligência Artificial

Paradigmas de Programação em Inteligência Artificial (PPROGIA)

2026/2027 · 1.º Semestre

Sistema Pericial de Apoio à Condução

Relatório Técnico

Motor de inferência, base de conhecimento, metaconhecimento e explicação em Prolog

Autores

Ana Salomé Ferreira Teixeira	1210803@isep.ipp.pt
Diogo Miguel Gonçalves Ribeiro	1260429@isep.ipp.pt
João Gabriel Domingos de Azevedo	1211519@isep.ipp.pt
Pedro Nuno Verde Cerqueira	1260443@isep.ipp.pt
Rúben Alexandre Abreu Freitas	1231267@isep.ipp.pt
Rúben Carlos Zamith Ribeiro	1161532@isep.ipp.pt

Docência

António Constantino Lopes Martins
José Antonio Reis Tavares
Jorge Alexandre Novo Meira

25 de outubro de 2026

Resumo

Este relatório descreve o motor de inferência e a base de conhecimento, em Prolog, de um sistema pericial de apoio à condução. A cada 100 ms, o sistema decide a velocidade e a faixa de um veículo simulado a partir do que os sensores observam: veículos à frente, peões, semáforos, sinais, perigos na via e meteorologia. A simulação está em Java e a interface em Angular, mas todas as decisões cabem ao motor de regras. O projeto contou com o apoio de um perito em robótica e sistemas autónomos.

Não se usou o motor da unidade curricular. O grupo desenvolveu em SWI-Prolog um motor de regras de produção próprio, com memória de trabalho, agenda, resolução de conflitos por precedência e refração. Combina encadeamento para a frente sobre os factos com encadeamento inverso dentro das condições. A base de conhecimento tem ∼120 regras, incluindo um controlador difuso para a meteorologia. Replicam, com os mesmos nomes e pela mesma ordem de disparo, as regras do segundo motor do sistema, em Drools.

O metaconhecimento (os tipos de facto que cada regra lê, a sua precedência e as instanciações que já dispararam) é derivado automaticamente das próprias regras e serve para controlar e acelerar a inferência. Depois de cada disparo, só se avaliam de novo as regras afetadas, o que reduz as avaliações de regras em 82,5 %.

Cada facto derivado e cada proposta guardam a regra e a evidência que os produziram. A partir desse registo, o sistema responde a três perguntas sobre uma decisão: *How* (como foi obtida), *Why* (porque foi tomada) e *Why not* (porque não foi tomada uma alternativa, ou porque não disparou uma regra).

Palavras-chave: sistemas periciais; Prolog; motor de inferência; regras de produção; metaconhecimento; explicabilidade; apoio à condução.

Índice

1 Perito **1**

2 Motor de inferência **1**
2.1 O motor de regras . . . 2
2.2 Comparação com o motor da unidade curricular . . . 3

3 Integração do motor Prolog com o Java **4**

4 Base de conhecimento **6**
4.1 O que a base contém . . . 6
4.2 Representação e organização . . . 6
4.3 Alguns exemplos . . . 7
4.4 Acesso e manutenção . . . 7

5 Metaconhecimento **8**
5.1 Definição e finalidade . . . 8
5.2 Representação no motor Prolog . . . 8
5.3 Índice de dependências . . . 9
5.4 Conhecimento de controlo . . . 9

6 Módulo de explicação **10**
6.1 O que fica registado . . . 10
6.2 How . . . 11
6.3 Why . . . 11
6.4 Why not . . . 12

7 Interface gráfica e arquitetura Web **12**

8 Dificuldades **13**

9 Conclusão **14**

Referências **15**

A Mapeamento entre código e relatório **16**

Lista de Figuras

2.1 Os dois motores alimentam a mesma arbitragem: só um está ativo em cada sessão, e ambos devolvem propostas, nunca ações. 2

6.1 Percurso da informação no módulo de explicação: a via em tempo real, a cada ciclo, e a via histórica, sobre a viagem gravada ao parar. 10

7.1 Interface Web com o motor Prolog ativo, durante uma ultrapassagem com chuva. 13

Lista de Tabelas

2.1 Motor da unidade curricular e motor do projeto 3

3.1 Pedidos JSON aceites pelo `server.pl` 5

4.1 Tipos de facto lidos pelas regras 6

5.1 Predicados de metaconhecimento do motor 8

5.2 Efeito do índice de dependências (14 cenários, 2 100 ciclos) 9

A.1 Localização no código de cada tema do relatório 16

1 | Perito

O perito selecionado para apoiar este projeto foi o Professor Doutor Agostinho Gil Teixeira Lopes, Professor Adjunto do Departamento de Engenharia Informática do Instituto Superior de Engenharia do Porto (ISEP). A sua formação e experiência abrangem áreas diretamente relacionadas com o projeto, nomeadamente robótica, sistemas autónomos, visão por computador, inteligência artificial, sensores, automação e controlo [1].

Concluiu a licenciatura em Informática e Matemáticas Aplicadas em 2002, na Universidade Portucalense, e o doutoramento em Engenharia Informática em 2006, na Cranfield University, no Reino Unido, com especialização em automação, controlo e robótica. Desenvolveu posteriormente atividades de investigação e pós-doutoramento relacionadas com a automatização e o controlo de processos industriais [1, 3].

Ao longo da sua carreira, exerceu funções de investigação e docência na Universidade do Minho e na Universidade da Maia, integrando o ISEP em outubro de 2024. A sua atividade científica inclui investigação em robótica móvel autónoma, visão artificial, redes neuronais, localização e navegação de robôs, bem como orientação de projetos académicos nestas áreas [1, 2, 4].

A sua experiência justifica a escolha como perito, uma vez que lhe permite avaliar a representação e interpretação dos dados sensoriais, os critérios de criticidade e as regras de tomada de decisão do sistema.

Neste sentido, a escolha deste perito justifica-se, sobretudo, pela proximidade entre as respetivas áreas de especialização e os desafios associados ao sistema pericial proposto. Embora o projeto não tenha como finalidade o desenvolvimento de um sistema de condução autónoma para utilização num veículo real, a representação do conhecimento necessária à análise dos cenários simulados depende da compreensão de como a informação deve ser interpretada e utilizada na avaliação de situações e na definição das ações realizadas.

Ao longo do projeto, a equipa realizou duas reuniões por videoconferência com o perito e trocou com ele várias mensagens de correio eletrónico, com perguntas e respostas. O contributo do perito foi particularmente pertinente na discussão de aspetos como, por exemplo, a relação entre velocidade, distância e capacidade de travagem, a identificação de situações críticas, o número de leituras necessário para confirmar uma deteção e o tratamento de informação contraditória. Estas questões encontram-se diretamente relacionadas com a aquisição e representação do conhecimento, uma vez que permitem fundamentar as condições, exceções e prioridades consideradas na construção da base de conhecimento. Assim, a participação do perito permitiu complementar a pesquisa realizada pela equipa, questionar pressupostos inicialmente definidos e orientar a formalização de conhecimento relevante para um sistema pericial cujas decisões possam ser compreendidas e analisadas pelos utilizadores.

No entanto, uma vez que o perito não é especialista em legislação rodoviária, nem em engenharia e homologação automóvel, o conhecimento normativo foi consultado diretamente no Código da Estrada [5].

2 | Motor de inferência

O motor de inferência é um motor de regras de produção desenvolvido em SWI-Prolog. Em cada ciclo de 100 ms recebe o estado do veículo e os eventos dos sensores, dispara as regras e devolve propostas de decisão, que a arbitragem em Java transforma numa única decisão (Figura 2.1). As secções seguintes descrevem as regras, a memória de trabalho e o ciclo do motor, e comparam-no com o motor da unidade curricular.

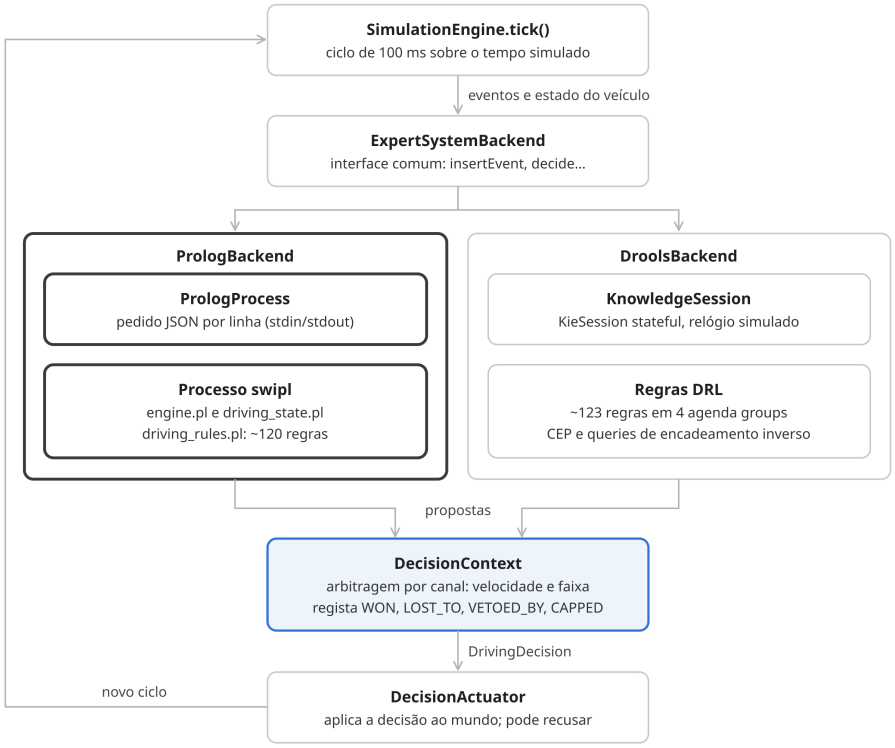


Figura 2.1: Os dois motores alimentam a mesma arbitragem: só um está ativo em cada sessão, e ambos devolvem propostas, nunca ações.

2.1 O motor de regras

As regras são dados: `rule(Id, Precedência, Condições, Ações)`. A Listagem 2.1 mostra um exemplo.

```
rule('Emergency stop for pedestrian extremely close',
     880,
     [ pedestrian_conflict(P),
       pedestrian(P, D),
       test(D =< 6.0) ],
     [ assert(proposal(emergency_brake, 0.0, critical, emergency_safety,
                       'Pedestrian critical distance in current path')) ]).
```

Listagem 2.1: Uma regra do motor Prolog (`driving_rules.pl`)

O segundo argumento, a precedência (880 na Listagem 2.1), só decide a ordem de disparo: entre as regras prontas a disparar, dispara primeiro a de número maior. Não se confunde com a prioridade da proposta (`emergency_safety`, 70), que decide qual das propostas vence na arbitragem (Capítulo 3); também aí vence o número maior.

Uma condição pode ser um padrão a unificar com um facto, `not(C)` (negação por falha), `test(Objetivo)` (aritmética ou um predicado auxiliar que lê os factos através de `known/1`) ou `prove(Objetivo)`, provado por encadeamento inverso (`prove_goal/2`). A única ação é `assert(Facto)`. O motor faz assim encadeamento para a frente sobre os factos e encadeamento inverso dentro das condições, por exemplo em `safe_to_change_lane/1`, cujos subobjetivos (`maneuver_idle`, `no_continuous_line`, `side_clear`, `room_to_follow`) descem até aos factos percebidos.

- **Memória de trabalho:**
  - `wm_fact(Id, Facto)` guarda os factos do ciclo, tanto os percebidos como os derivados;
  - `wm_support(Id, Regra, Evidência)` guarda a justificação de cada facto derivado: a regra que o concluiu, os identificadores dos factos em que assentou e os padrões negados que se verificaram. Um facto sem suporte veio da perceção.

- **Ciclo:** `run_rules/0` avalia todas as regras e enche a agenda. Depois, até a agenda esvaziar, dispara a melhor ativação, escolhida pela resolução de conflitos; a refração impede que uma instanciação dispare duas vezes no mesmo ciclo (Secção 5.4). Os factos nunca são retirados dentro de um ciclo.
- **Estado entre ciclos:** fica em `driving_state.pl`, que guarda as últimas leituras, a janela de TTC, o compromisso perante o semáforo, o cronómetro do STOP e o estado da ultrapassagem. No início de cada ciclo carrega tudo como factos (`load_cycle_facts/0`). As regras não alteram estado: concluem o que mudou (por exemplo `overtake_started/3`), e `store_conclusions/0` grava-o no fim do ciclo.

2.2 Comparação com o motor da unidade curricular

O sistema não usa o motor da unidade curricular (`sp_exp1.pl`, `sp_exp2.pl` [6]): o motor (`engine.pl` [7]) foi desenvolvido pelo grupo. O problema exige decidir a cada 100 ms sobre uma memória de trabalho que muda em cada ciclo, disparar as regras pela mesma ordem que o Drools e devolver explicações como dados ao Java, e não como texto na consola. Do motor da UC manteve-se a ideia central: registar a justificação de cada facto quando é concluído (`justifica/3` corresponde ao nosso `wm_support/3`) e explicar a partir desse registo. As restantes diferenças respondem a pontos que a própria aula 6 [8] deixa em aberto (Tabela 2.1).

Tabela 2.1: Motor da unidade curricular e motor do projeto

Aspeto	**`sp_exp2.pl`**	**`engine.pl`**
Regras	`regra N se [...] entao [...]`, com operadores e um número	Termos `rule(Id, Precedência, Condições, Ações)`, identificados por um nome descritivo
Condições	Factos, `nao` e `avalia/1`, que compara um valor com uma constante	Padrões, `not/1` sobre padrões e testes, `test/1` (qualquer objetivo) e `prove/1` (encadeamento inverso)
Controlo	Dispara uma regra assim que um facto a satisfaz, pela ordem do *backtracking* sobre factos e regras	Agenda com resolução de conflitos, a começar pela precedência (Secção 5.4)
Repetições	`cria_facto` só verifica se a conclusão já existe, e não se o par facto–regra já disparou [8, diap. 39]	Refração por regra e instanciação (`fired/2`); um facto repetido é ignorado
Metaconhecimento	`facto_dispara_regras/2` escrito à mão em `veiculos2.txt`; gerá-lo é o exercício proposto [8, diap. 47–50]	Derivado das regras no arranque (`rule_reads/2`), incluindo os factos lidos pelos predicados auxiliares
Disjunção	Duas regras com a mesma cabeça, e não um OU dentro da regra [9, p. 8]	Igual: `pedestrian_conflict/1` tem uma regra por ramo (Listagem 2.3)
Explicações	`como/1` e `whynot/1` escrevem na consola; `whynot/1` parte de um facto	`explain/2` e `why_not/2` devolvem termos; `why_not/2` parte de uma regra (Capítulo 6)

Três destas diferenças merecem nota. A primeira é o metaconhecimento. A aula 6 mostra como gerar `facto_dispara_regras/2` a partir de uma lista de termos, mas deixa às equipas a obtenção automática dessa lista. `ensure_index/0` obtém-na de todas as regras e segue também os predicados auxiliares que as condições chamam, onde estão muitas das dependências (Secção 5.3). A segunda é a refração: o pseudocódigo da aula pede que cada par facto–regra dispare uma vez, mas o código só impede conclusões repetidas. No nosso motor é cada instanciação de cada regra que dispara no máximo uma vez por ciclo. A terceira é a resolução de conflitos, que a UC não trata (Secção 5.4).

As condições do nosso motor exprimem mais do que factos e constantes. A Listagem 2.2, a regra mais longa da base, aplica o Art. 13 do Código da Estrada (circular pela faixa mais à direita). Usa as cinco formas de condição: padrões (`ego_lane(Lane)`), testes aritméticos (`test(Now >= 4000)`), padrões negados (`not(obstacle(_, _, _, _))`), testes negados sobre predicados auxiliares (`not(test(front_vehicle_within(80.0)))`) e um subobjetivo provado por encadeamento inverso

(`prove(no_continuous_line(right))`). Só propõe voltar à direita quando nada mais exige atenção: nenhum obstáculo, buraco, barreira ou peão em conflito, nenhum veículo a menos de 80 m e nenhuma ultrapassagem ou mudança de faixa pedida pelo condutor.

```
rule('Enforce keep right road law when right lane is safe',
     460,
     [ ego_lane(Lane),
       test(Lane \== right),
       not(presentation_demo),
       lane_change_state(idle),
       prove(no_continuous_line(right)),
       not(right_lane_blocked_ahead),
       time_ms(Now),
       test(Now >= 4000),
       lane_occupancy(right, false, RightFront),
       test(RightFront > 80.0),
       test(fresh_side_reading(right)),
       not(move_off_state(true, _)),
       not(obstacle(_, _, _, _)),
       not(pothole(_, _, _, _)),
       not(barrier(_, _, false, _)),
       not(pedestrian_conflict(_)),
       not(test(front_vehicle_within(80.0))),
       not(lane_change_request(_)),
       not(test(driver_lane_change_hold(Now))),
       not(test(current_overtaking(true, _, _, _, _))) ],
     [ assert(proposal(return_right, no_target, low, maneuver_safety,
                       'Portuguese Highway Code Art. 13: Drive on the rightmost available
    ↪ safe lane')) ]).
```

Listagem 2.2: Uma das regras mais longa da base de conhecimento (`driving_rules.pl`)

A condição `not(pedestrian_conflict(_))` lê uma conclusão intermédia, a disjunção "peão na trajetória ou a atravessar". Como a UC aconselha, a disjunção é uma conclusão própria, com uma regra por ramo (Listagem 2.3). A precedência 1000 garante que o conflito é conhecido antes de qualquer regra que o leia, e a explicação mostra qual dos ramos se verificou.

```
rule('Pedestrian conflicts with trajectory (in path)',
     1000,
     [ pedestrian_in_path(P) ],
     [ assert(pedestrian_conflict(P)) ]).

rule('Pedestrian conflicts with trajectory (crossing)',
     1000,
     [ crossing_state(P, crossing) ],
     [ assert(pedestrian_conflict(P)) ]).
```

Listagem 2.3: Uma disjunção como duas regras (`driving_rules.pl`)

3 | Integração do motor Prolog com o Java

`ExpertSystemBackend` define o ciclo comum aos motores: avançar o relógio, iniciar o ciclo, inserir eventos, atualizar o veículo, decidir e expor a memória e a explicação. `PrologBackend` implementa esse contrato e permite trocar o motor sem modificar o `SimulationEngine`.

Comunicação. O motor corre num processo `swipl` persistente, gerido pela classe `PrologProcess`. O Java copia os ficheiros Prolog para uma pasta de trabalho e arranca o processo com `swipl -q -g main -t halt server.pl`. O arranque acontece quando o motor é iniciado, num RESET ou numa troca de motor. Os dois lados trocam mensagens JSON pela entrada e saída padrão: um pedido por linha, e exatamente uma resposta por linha, mesmo quando o pedido falha. Lançar o interpretador custa cerca de 150 a 250 ms, o que é incompatível com uma decisão a cada 100 ms. Além disso, o lado

Prolog tem de guardar estado entre ciclos. Por isso o processo é mantido vivo durante toda a sessão. O pedido `reset` inicia a sessão com a velocidade de cruzeiro do cenário e com as constantes de paragem e as prioridades definidas no Java, para que não haja valores duplicados nos dois lados. A Tabela 3.1 resume as mensagens JSON.

Tabela 3.1: Pedidos JSON aceites pelo `server.pl`

Comando	Entrada principal	Resposta
`tick`	Veículo, eventos e decisão anterior	Propostas, factos, limite, verificações e métricas
`reset`	Cruzeiro, parâmetros e prioridades	Confirmação e reinicialização do estado
`whynot`	Identificador da regra	Condições que impediram a correspondência
`dependencyIndex`	Booleano	Estado do índice para *benchmark*

O `server.pl` lê uma linha, converte o JSON num *dict* Prolog [10] e responde sempre com outra linha. O *stdout* contém apenas respostas. Eventuais mensagens de aviso ou erro seguem para o *stderr*. O *flush* explícito evita que o Java fique à espera de dados retidos no *buffer*.

Pedidos e respostas. Durante o ciclo, o `PrologBackend` acumula os eventos dos sensores e os comandos do condutor. Preenche também um `VehicleInput` com o que o Drools lê do facto do veículo: velocidade, posição, faixa, estado da manobra e faixa de destino, marcações da estrada, segurança das faixas adjacentes (calculada pela perceção em Java) e se há um perigo à frente na faixa da direita. Tudo segue num único pedido `tick`, com a decisão anterior (ação, regra e prioridade). A resposta traz:

- as propostas, cada uma com a ação, a velocidade, o risco, a prioridade, a razão, a regra e a evidência;
- os factos do ciclo;
- o limite de velocidade em vigor;
- os subobjetivos da segurança de mudança de faixa nos dois lados;
- as métricas do motor.

Arbitragem e aplicação. O Java não decide nada por si: entrega as propostas ao `DecisionContext`. A arbitragem trata em separado a velocidade e a faixa.

- Na velocidade, as emergências e as paragens obrigatórias (prioridade 60 ou mais) sobrepõem-se a tudo. Entre elas vence a mais prioritária e, em empate, a mais lenta. Abaixo disso, cada teto de velocidade que se verifica limita a velocidade de base, e decide o mais baixo, se ficar abaixo dessa base.
- Na faixa, vence a maior prioridade. Em empate, regressar à direita vence mudar de faixa.
- Uma mudança de faixa só é vetada quando uma paragem de prioridade superior está de facto a travar o veículo.
- Sem propostas, o veículo mantém a velocidade e a faixa.

Cada proposta fica com o seu resultado: venceu, perdeu, foi vetada ou foi limitada. O `DecisionActuator` aplica a decisão ao veículo. Pode recusar uma mudança de faixa que não consiga executar, e essa recusa fica registada. O ciclo é então guardado para o módulo de explicação.

4 | Base de conhecimento

A base de conhecimento (`driving_rules.pl`) é um conjunto de ∼120 regras de produção sobre condução, que replicam as regras de decisão do motor Drools do sistema.

4.1 O que a base contém

O objetivo é que o comportamento do veículo fique descrito nas regras, e não espalhado pelo código de controlo. A base cobre emergências, peões, veículo da frente e ultrapassagens, limites de velocidade, STOP, semáforos, perigos na via, mudanças de faixa, barreiras e meteorologia. Algumas regras seguem diretamente o Código da Estrada [5], como o Art. 13 (circular pela direita, Listagem 2.2).

As regras leem 66 tipos de facto, que se agrupam como mostra a Tabela 4.1.

Tabela 4.1: Tipos de facto lidos pelas regras

Tipo	Exemplos	Origem
Perceção	`ego_speed(Kmh)`, `pothole(Id, DistanceM, Severity, Lane)`, `traffic_light(Id, red|yellow|green, DistanceM)`, `lane_occupancy(Side, Occupied, FrontDistanceM)`	Sensores e veículo, carregados por `driving_state.pl` no início do ciclo
Estado anterior	`overtaking/5`, `light_commitment/2`, `stop_hold_started/2`, `previous_decision/2`	Estado guardado entre ciclos (38 tipos) e a decisão anterior
Conclusões intermédias	`pedestrian_conflict(Id)`, `fuzzy_membership/3`, `fuzzy_conclusion/2`, `overtake_started/3`	Concluídas por regras no próprio ciclo
Propostas	`proposal(Action, SpeedKmh, Risk, Priority, Reason)`	Concluídas por regras, arbitradas no Java

Além das regras e dos factos, a base tem predicados auxiliares, que leem os factos através de `known/1` (por exemplo `front_vehicle_within/1`), e subobjetivos provados por encadeamento inverso (Secção 4.3). As constantes numéricas, como as distâncias de paragem e o valor de cada prioridade, não estão escritas no Prolog: chegam do Java no pedido `reset` (`param/2`, `priority_value/2`).

4.2 Representação e organização

No Prolog, regras e factos são termos, como `rule/4` ou `traffic_light(Id, red|yellow|green, DistanceM)`, e as conclusões de decisão são factos `proposal/5`. A Listagem 4.1 mostra uma regra da base.

```
1 rule('Reduce speed when exceeding detected limit',
2      300,
3      [ ego_speed(V),
4        speed_limit(Limit),
5        test(V > Limit + 0.1) ],
6      [ assert(proposal(reduce_speed, Limit, moderate, speed_compliance,
7                        'Vehicle exceeds current legal speed limit')) ]).
```

Listagem 4.1: Regra de excesso de velocidade em Prolog (`driving_rules.pl`)

Ou seja, se a velocidade passar o limite em vigor em mais de 0,1 km/h, propõe-se reduzir para o limite, com risco moderado e prioridade de cumprimento de velocidade (20).

A perceção não tem regras Prolog, porque é tratada em `driving_state.pl`.

4.3 Alguns exemplos

Limite de velocidade. A câmara deteta o sinal até 120 m à frente. O `driving_state.pl` guarda-o como pendente, com a sua posição na estrada, e carrega-o como `pending_speed_limit(Kmh, SignPositionM)`. Enquanto o sinal está à frente, `'Slow down in time for the speed limit ahead'` faz o veículo chegar a ele já ao novo limite. Quando o veículo chega ao sinal, o limite pendente passa a `speed_limit(Kmh)`, e a regra da Listagem 4.1 pode propor `reduce_speed`. Os dois limites estão separados porque, aplicado logo que o sinal era visto, o limite fazia o veículo abrandar demasiado cedo.

Meteorologia. Para a meteorologia recorre-se a lógica difusa: um controlador de Mamdani com 8 regras, em três níveis de precedência. Duas regras de fuzificação (360) convertem a visibilidade e a aderência de `weather/3` em graus de pertença, `fuzzy_membership(Variable, Term, Degree)`. Cinco regras de inferência (355) combinam-nos, com E como mínimo e OU como máximo, e concluem `fuzzy_conclusion(Term, Degree)` (Listagem 4.2). Uma regra de desfuzificação (350) agrega as conclusões e calcula o centroide, que é a velocidade recomendada: um único teto `reduce_speed` com prioridade de perigo na via (40). A precedência garante que cada nível só dispara depois do anterior.

```
rule('Fuzzy: poor visibility or poor grip -> very slow',
     355,
     [ fuzzy_membership(visibility, poor, V),
       fuzzy_membership(grip, poor, G),
       test(( D is max(V, G), D > 0.0 )) ],
     [ assert(fuzzy_conclusion(very_slow, D)) ]).
```

Listagem 4.2: Uma regra de inferência difusa (`driving_rules.pl`)

Mudança de faixa segura. `safe_to_change_lane/1` é um objetivo, provado por encadeamento inverso a partir de quatro subobjetivos (Listagem 4.3). As regras de desvio chamam-no nas condições com `prove(safe_to_change_lane(left))`. Mudar de faixa é seguro se não houver outra manobra em curso nem linha contínua, se o ângulo morto estiver livre e se houver espaço para seguir o veículo dessa faixa, com 12 m de folga (`StoppingPolicy.roomToFollow`).

```
safe_to_change_lane(Side) :-
    maneuver_idle,
    no_continuous_line(Side),
    side_clear(Side),
    room_to_follow(Side).
```

Listagem 4.3: Objetivo provado por encadeamento inverso (`driving_rules.pl`)

4.4 Acesso e manutenção

Os padrões das regras são unificados com `wm_fact/2`, e os auxiliares leem factos com `known/1`. Dentro de um ciclo, os factos nunca são retirados: não há manutenção de verdade. As crenças mantêm-se atualizadas entre ciclos, porque cada ciclo começa com os factos carregados de novo a partir do estado em `driving_state.pl`. Esse estado esquece o que deixou de ser observado (`expire_old_readings/1`): por exemplo, um obstáculo, um buraco ou uma barreira desaparecem 1 s depois da última leitura. As regras em si não mudam durante a execução. A manutenção faz-se editando os ficheiros, e o `PrologDroolsEquivalenceTest` confirma que as duas bases continuam coerentes.

5 | Metaconhecimento

5.1 Definição e finalidade

Neste projeto, metaconhecimento é o conhecimento sobre as próprias regras, e não sobre a condução em si, usado para controlar e acelerar a inferência. Saber que se deve reduzir a velocidade quando o limite é ultrapassado é conhecimento do domínio. Saber que a regra que o faz lê os factos `ego_speed/1` e `speed_limit/1`, que tem precedência 300 e que já disparou neste ciclo é metaconhecimento. É esta a definição que está no cabeçalho de `engine.pl`, onde cada parte do metaconhecimento está assinalada com comentários "Meta-conhecimento" ou "Meta-conhecimento dinâmico".

O metaconhecimento resolve três problemas de controlo do motor de inferência:

- **resolução de conflitos**: quando várias regras estão prontas a disparar, qual dispara primeiro;
- **terminação**: a mesma instanciação de uma regra não pode disparar duas vezes no mesmo ciclo;
- **eficiência**: depois de cada disparo, só se avaliam de novo as regras que o facto acabado de concluir pode afetar.

5.2 Representação no motor Prolog

O metaconhecimento está em predicados dinâmicos de `engine.pl`, declarados com `:- dynamic` [11] e separados da memória de trabalho (`wm_fact/2`, `wm_support/3`, etc.), cujos predicados têm o prefixo `wm_` (Tabela 5.1).

Tabela 5.1: Predicados de metaconhecimento do motor

Predicado	Conhecimento representado	Utilização
`rule_reads(Nome/Aridade, Regra)`	Tipos de facto que cada regra lê	Avaliar de novo só as regras afetadas (`fire/2`, `rematch/1`)
`rule_precedence(Regra, P)`	Precedência de cada regra	Escolher a ativação seguinte (`best_activation/2`)
`rule_rank(Regra, N)`	Posição da regra na base de conhecimento	Desempate determinístico entre regras com a mesma precedência
`fired(Regra, Condições)`	Instanciações que já dispararam no ciclo	Refração (`already_fired/2`)

Este metaconhecimento é dinâmico, e não escrito à mão, em dois sentidos:

1. **É derivado das regras durante a execução.** Quando o motor arranca, `ensure_index/0` lê as próprias regras como dados e calcula `rule_reads/2`, `rule_precedence/2` e `rule_rank/2`, que guarda com `assertz` [12]. Quando uma regra é acrescentada ou alterada, o metaconhecimento acompanha-a no arranque seguinte, sem manutenção manual. O mesmo passo verifica que não há duas regras com o mesmo identificador.
2. **Muda durante a inferência.** `fired/2` ganha uma entrada a cada disparo e é apagado no início de cada ciclo.

5.3 Índice de dependências

O índice (`rule_reads/2`) regista, para cada regra, os tipos de facto (`Nome/Aridade`) de que depende. `ensure_index/0` obtém-nos por análise estática da regra:

- cada padrão e cada padrão negado (`not/1`) contam com o seu `Nome/Aridade`;
- em `test/1` e `prove/1`, contam os factos lidos com `known/1`, no próprio objetivo e nas cláusulas dos predicados auxiliares que ele chama, seguidos transitivamente com `clause/2`.

Com a base de conhecimento atual, o índice tem 603 entradas, para 120 regras e 66 tipos de facto. A regra `'Avoid pothole in centre lane by changing right when right is clear'`, por exemplo, depende de nove tipos de facto, mas só três aparecem nas suas condições (`ego_lane/1`, `pothole/4` e `ego_speed/1`). Os outros seis, como as marcações da estrada, a ocupação das faixas e o estado da manobra, vêm dos predicados auxiliares, sobretudo de `safe_to_change_lane/1`, chamado através de `prove/1`. Um índice feito só com os padrões visíveis da regra não veria estas dependências.

Durante a inferência, `fire/2` recolhe os tipos dos factos novos que o disparo acrescentou e, através de `rule_reads/2`, avalia de novo apenas as regras que os leem (`rematch/1`). As ativações dessas regras aparecem, mudam ou são canceladas antes do disparo seguinte, por exemplo quando uma condição `not(proposal(...))` deixa de se verificar. Sem o índice, todas as regras teriam de ser avaliadas de novo depois de cada disparo.

O teste `DependencyIndexBenchmarkTest` mede o efeito do índice. Corre os 14 cenários da biblioteca durante 150 ciclos cada (2 100 ciclos) em dois processos Prolog lado a lado, um com o índice e outro sem ele. Em todos os ciclos, as duas execuções tomaram a mesma decisão e dispararam as mesmas regras (10 231 disparos): o índice só muda o custo, que o próprio motor mede (`cycle_stats/3`). Com o índice, a inferência ocupa em média cerca de 1,5 % do ciclo de 100 ms (Tabela 5.2). Os tempos cobrem só a inferência, sem a comunicação JSON, e foram medidos numa única máquina (Intel Core Ultra 7 258V, SWI-Prolog 9.2.9).

Tabela 5.2: Efeito do índice de dependências (14 cenários, 2 100 ciclos)

	Com índice	Sem índice	Redução
Avaliações de regras, total (por ciclo)	258 894 (123)	1 479 720 (705)	82,5 %
Tempo médio por ciclo	1,47 ms	6,41 ms	77,0 %
Pior cenário (S10, chuva)	2,64 ms	9,62 ms	72,6 %

5.4 Conhecimento de controlo

Entre as ativações prontas, `best_activation/2` escolhe a de maior precedência (`rule_precedence/2`). Em empate, escolhe a da regra declarada primeiro (`rule_rank/2`) e, depois, a correspondência encontrada primeiro. A precedência de cada regra é a *salience* da regra Drools que ela replica, para que os dois motores do sistema disparem pela mesma ordem. As únicas exceções são as duas regras intermédias de peões, que só existem em Prolog (`'Pedestrian conflicts with trajectory (in path)'` e `'(crossing)'`). Estas têm precedência 1000, para concluírem `pedestrian_conflict/1` antes de qualquer regra que o leia.

A refração usa `fired/2`. Antes de uma instanciação entrar na agenda, `already_fired/2` compara as suas condições com as das instanciações que já dispararam, por variante (`=@=`), e deixa de fora as repetidas. Como os factos nunca são retirados dentro de um ciclo, isto impede que a mesma conclusão seja produzida repetidamente. Um limite de 5000 disparos por ciclo (`max_firings_per_cycle/1`) protege ainda contra uma base de regras que não pare de gerar instanciações novas, devolvendo um erro em vez de bloquear a simulação.

6 | Módulo de explicação

As explicações assentam num princípio: a justificação de cada conclusão é registada no momento da inferência, e não reconstruída depois. É a ideia que se manteve do motor da unidade curricular (Secção 2.2), aplicada a dois níveis. No motor Prolog, cada facto guarda como foi obtido. Na aplicação, cada ciclo de decisão guarda as propostas das regras e o resultado da arbitragem. A interface nunca deduz condições nem volta a avaliar regras: mostra apenas o que o motor registou. Por isso as explicações funcionam da mesma forma com o Prolog e com o Drools.

6.1 O que fica registado

Em cada ciclo, o motor Prolog regista:

- para cada facto derivado, `wm_support(Id, Regra, Evidência)`: a regra que o concluiu, os factos em que assentou e os padrões negados que se verificaram;
- para cada proposta, a ação, a velocidade, o risco, a prioridade, a razão, a regra e a evidência. A evidência é a lista desses factos, e cada facto derivado indica a regra que o concluiu, por exemplo `pedestrian_conflict(p1) (by rule '...')`;
- os quatro subobjetivos de `safe_to_change_lane/1` (`maneuver_idle`, `no_continuous_line`, `side_clear`, `room_to_follow`), provados um a um nos dois lados por `subgoal_results/2`, mesmo quando nenhuma regra propõe mudar de faixa;
- uma entrada na linha temporal por cada leitura dos sensores (`FACT INSERTED`) e por cada regra que concluiu algo (`RULE FIRED`).

A camada Java junta a cada proposta o seu resultado na arbitragem: venceu, perdeu para outra, foi vetada por uma paragem obrigatória, foi limitada por um teto de velocidade, ou foi recusada pelo atuador. Junta também um detalhe que identifica a regra vencedora ou o motivo da recusa.



Figura 6.1: Percurso da informação no módulo de explicação: a via em tempo real, a cada ciclo, e a via histórica, sobre a viagem gravada ao parar.

As duas vias da Figura 6.1 partem do mesmo registo. Na via de tempo real, o estado de cada ciclo chega à interface pelo WebSocket. O painel de decisão mostra cada proposta com o seu resultado na arbitragem, e outros painéis mostram a razão, a regra, a evidência, a linha temporal e a memória de

trabalho. Na via histórica, guarda-se um registo completo do ciclo (`SessionSnapshot`) sempre que a decisão muda. Quando o utilizador carrega no botão "STOP" da interface Web, abre-se um diálogo em que pode percorrer as decisões registadas. Para cada uma, a interface permite consultar os três tipos de explicação, *How*, *Why* e *Why not* (ver também o Capítulo 7).

Exemplo. Nas secções seguintes usa-se a mesma situação, executada no motor Prolog. O veículo segue a 50 km/h na faixa da direita, há um buraco na mesma faixa 40 m à frente, e a faixa da esquerda está livre. A regra `'Avoid pothole in right lane by changing left'` (prioridade 40) propõe `CHANGE_LANE_LEFT`, e a regra de base mantém a velocidade.

6.2 How

No motor, `explain/2` percorre as justificações `wm_support/3` recursivamente e devolve a árvore de derivação como um termo. Cada facto derivado é um nó `derived(Facto, Regra, Porque)`, cada facto percebido é uma folha `perceived(Facto)`, e cada padrão negado é `absent(Padrão)`. A Listagem 6.1 mostra a árvore da proposta do exemplo.

```
derived(proposal(change_lane_left, no_target, low, road_hazard,
                 'Avoiding pothole by moving into the clear left lane'),
        'Avoid pothole in right lane by changing left',
        [ perceived(ego_lane(right)),       perceived(pothole(p1,40,high,right)),
          perceived(ego_speed(50)),         perceived(lane_change_state(idle)),
          perceived(right_marking(dashed)), perceived(lane_occupancy(left,false,inf)) ])
```

Listagem 6.1: Árvore de derivação devolvida por `explain/2` para a proposta do exemplo

A árvore tem seis factos, embora as condições da regra só nomeiem três (`ego_lane`, `pothole` e `ego_speed`). Os outros três vêm da prova de `safe_to_change_lane(left)`, feita por encadeamento inverso (`prove/1`): a prova regista os factos que lê com `known/1`, e estes passam a fazer parte da evidência. Quando a decisão depende de conclusões intermédias, a árvore desce por elas. Numa travagem de emergência por um peão, por exemplo, a proposta assenta em `pedestrian_conflict(p1)`, que é ele próprio um nó `derived(...)` com a regra que o concluiu e os factos percebidos em que esta assentou.

Na aplicação, o *How* mostra o caminho que o ciclo escolhido percorreu no fluxograma de decisão, um para a velocidade e outro para a faixa. Cada pergunta do fluxograma está associada às regras que lhe respondem. Segue o ramo "sim" quando uma dessas regras fez uma proposta nesse ciclo, e o ramo "não" caso contrário. O caminho termina onde a arbitragem fixou a decisão. Na velocidade, isso acontece numa paragem de emergência ou obrigatória que venceu. Os tetos de velocidade aplicam-se todos, por isso o caminho passa por eles até ao mais baixo. Na faixa, o caminho termina na proposta que venceu. As perguntas seguintes aparecem fora do caminho, com o que as suas regras tenham proposto.

6.3 Why

Nos materiais da unidade curricular, o *Why* é o do MYCIN: quando o motor pergunta um facto ao utilizador, este pode perguntar porquê, e o motor mostra a regra que está a tentar provar (o `sp_exp2.pl` não o implementa) [13]. Este sistema não faz perguntas ao utilizador. Todos os factos vêm dos sensores e do estado do veículo, dez vezes por segundo, um ritmo a que nenhum condutor conseguiria responder. Por isso, aqui, o *Why* justifica uma decisão já tomada: responde porque é que o veículo fez o que fez. Para cada canal, a aplicação junta a regra que decidiu, a sua razão, a evidência e o resultado da arbitragem, bem como as outras propostas que também foram consideradas. No canal da faixa, uma mudança de faixa inclui ainda os subobjetivos de segurança desse lado. No exemplo, o veículo muda para a esquerda porque `'Avoid pothole in right lane by changing left'` se verificou, com a razão "Avoiding pothole by moving into the clear left lane". A evidência são os seis factos acima, e os quatro subobjetivos de `safe_to_change_lane(left)` verificaram-se. A velocidade mantém-se porque nenhuma regra encontrou motivo para acelerar ou abrandar.

6.4 Why not

O *Why not* responde porque é que o veículo não fez outra coisa. Na interface, o utilizador pergunta por uma alternativa, por exemplo "porque não abrandou para o buraco?", e a resposta vem de dois mecanismos. Um terceiro, no motor, desce até à condição exata de uma regra:

- **Uma alternativa não escolhida.** Para cada alternativa que o ciclo não seguiu, a aplicação explica porquê: acelerar, manter ou abrandar na velocidade, e manter a faixa, mudar para a esquerda ou para a direita, ou regressar à direita na faixa. Cada alternativa é classificada a partir do registo do ciclo. Ou foi proposta mas perdeu na arbitragem (e a resposta diz para que regra, e por que princípio), ou nunca foi proposta porque as suas condições não se verificaram, ou não se aplica naquele estado, ou o ciclo não registou o suficiente. No último caso, a resposta di-lo em vez de adivinhar. No exemplo, "porque não abrandou?" tem como resposta que nenhuma regra encontrou uma condição que exigisse abrandar, e "porque não mudou para a direita?" não se aplica, porque o veículo já está na faixa mais à direita.
- **Uma mudança de faixa recusada.** Os subobjetivos de `safe_to_change_lane/1` registados em cada ciclo dizem qual deles falhou. Em vez de uma mensagem genérica como "mudança impossível", a explicação indica a causa: outra manobra em curso, linha contínua, ângulo morto ocupado, ou falta de distância para seguir o veículo da outra faixa.
- **Uma regra que não disparou.** No motor, `why_not/2` procura, entre todas as correspondências parciais das condições da regra, da esquerda para a direita, a que avançou mais. Devolve a condição onde parou, com as ligações dessa correspondência, seguida das condições posteriores que também falham. Chega-se a ele pelo pedido `whynot` do `server.pl`. No exemplo, para a regra que abranda perante um buraco, aponta a condição negada que falhou: o buraco ainda pode ser contornado (Listagem 6.2).

```
{"cmd":"whynot", "rule":"Pothole ahead reduce speed"}
{"ok":true, "rule":"Pothole ahead reduce speed",
 "failedConditions":["not(test(pothole_avoidable_now(50,40)))"]}
```

Listagem 6.2: Pedido `whynot` e resposta do `server.pl` no exemplo

7 | Interface gráfica e arquitetura Web

A aplicação segue uma arquitetura cliente-servidor. O servidor, em Java 17 (ou superior) com Spring Boot 3.3.4, corre a simulação e o motor de inferência. O cliente é uma aplicação Angular 18 que só apresenta o estado e envia comandos. A interface nunca avalia condições de condução: as distâncias de segurança, os limites de velocidade e a segurança das mudanças de faixa são decididos apenas no motor de regras. A interface limita-se a desenhar as coordenadas que o servidor envia.

A comunicação usa dois canais. Os comandos seguem por REST, cada um com a sua resposta: iniciar, parar ou reiniciar a simulação, escolher um cenário, acrescentar elementos à estrada e trocar de motor. O estado da simulação chega por um WebSocket (`/api/ws/simulation`). O servidor publica um *snapshot* em JSON a cada ciclo de 100 ms, que é o mesmo conteúdo que a API REST devolve. Se a ligação cair ou deixar de receber dados, o cliente volta a ligar-se.

A interface (Figura 7.1) mostra a estrada em perspetiva, com o veículo, os outros atores, a sinalização e a meteorologia. Mostra também a decisão atual e as propostas concorrentes, cada uma com o seu resultado na arbitragem. Os restantes painéis apresentam a regra de origem, a razão e a evidência, a linha temporal das regras e dos factos, a memória de trabalho e as leituras dos sensores.

Uma biblioteca de cenários e uma caixa de ferramentas permitem montar situações durante a execução: veículos, peões, sinais, semáforos, buracos, obstáculos, barreiras e chuva. Um seletor troca entre os motores Drools e Prolog, recarregando o cenário. Se o motor falhar, um aviso mostra a mensagem de erro. O diálogo de perguntas *Why*, *Why not* e *How*, que abre quando a simulação para (usando o botão "STOP"), é descrito na Secção 6.1.



Figura 7.1: Interface Web com o motor Prolog ativo, durante uma ultrapassagem com chuva.

8 | Dificuldades

A maior parte das dificuldades surgiu ao reproduzir em Prolog o que o Drools oferece diretamente, e não ao escrever as regras de condução:

- **Estado entre ciclos.** O Drools tem memória com estado, eventos que expiram, janelas temporais, *salience* e reação a atualizações. Em Prolog foi preciso separar o estado persistente (`driving_state.pl`) da memória de cada ciclo, representar as janelas temporais com *timestamps* e escrever um motor com agenda explícita (Capítulo 2).
- **Ordem dos eventos.** Os eventos de um ciclo podem chegar em qualquer ordem. O `driving_state.pl` trata primeiro as distâncias frontais, para que a velocidade relativa e o TTC atualizem o veículo da frente correto, seja qual for a ordem no JSON.
- **Factos lidos por predicados auxiliares.** Um `test/1` não deixa ver, na explicação, os factos que os auxiliares consultam. A forma `prove/1` prova o objetivo por encadeamento inverso e regista esses factos, o que permite explicar, por exemplo, porque é que uma mudança de faixa não é segura (Secção 6.2).
- **Velocidade e faixa.** Com uma só ordenação, uma mudança de faixa competiria com uma travagem, embora atuem em dimensões diferentes. O `DecisionContext` arbitra as duas em separado e só veta a manobra quando uma paragem mais prioritária está a travar o veículo (Capítulo 3).

9 | Conclusão

O grupo desenvolveu em SWI-Prolog um motor de regras de produção próprio e integrou-o num sistema pericial que decide a velocidade e a faixa de um veículo simulado a cada 100 ms. O motor (`engine.pl`) torna explícitos mecanismos que noutros motores ficam internos: a agenda com resolução de conflitos, a refração, a justificação de cada facto e o índice de dependências. Resolve também dois pontos que a aula 6 deixa em aberto: o metaconhecimento é gerado a partir das próprias regras, e cada instanciação de uma regra dispara no máximo uma vez por ciclo.

A base de conhecimento tem ∼120 regras e replica, com os mesmos nomes e pela mesma ordem de disparo, as regras do motor Drools do sistema. Como cada facto derivado e cada proposta guardam a regra e a evidência que os produziram, as explicações *How*, *Why* e *Why not* são construídas a partir do que o motor registou, e não reconstruídas depois.

O sistema tem limitações:

- não faz perguntas ao utilizador, porque todos os factos vêm dos sensores (Secção 6.3);
- o conhecimento só foi exercitado no simulador, pelo que o sistema é um protótipo académico de apoio à decisão, e não um sistema para um veículo real.

Referências

[1] Ciência Vitae. Agostinho Gil Teixeira Lopes — Curriculum Vitae. https://www.cienciavitae.pt/portal/pt/F91B-B8E9-3D47. Consultado em 20 de outubro de 2026.

[2] Instituto Superior de Engenharia do Porto. Contrato de trabalho em funções públicas por tempo indeterminado — Agostinho Gil Teixeira Lopes, Professor Adjunto. 2 de outubro de 2024. https://fileserver.portal.isep.ipp.pt/DownFile.aspx?key=e10334f5-712e-40af-affe-4c185b480d92. Consultado em 20 de outubro de 2026.

[3] A. G. T. Lopes. *Arc-Based Sensing in Narrow Groove Pipe Welding*. Tese de doutoramento, Cranfield University, 2006. https://dspace.lib.cranfield.ac.uk/bitstream/1826/1514/1/Lopes-2006.pdf. Consultado em 20 de outubro de 2026.

[4] Universidade do Minho, Departamento de Eletrónica Industrial. Gestão de Dissertações: orientações de Agostinho Gil Teixeira Lopes. https://intranet.dei.uminho.pt/gdmi/en/site/teses/. Consultado em 20 de outubro de 2026.

[5] Portugal. Código da Estrada, aprovado pelo Decreto-Lei n.º 114/94, de 3 de maio, na redação em vigor. Diário da República. https://diariodarepublica.pt/dr/detalhe/decreto-lei/114-1994-250659. Consultado em 21 de outubro de 2026.

[6] PPROGIA. Ficheiros de apoio a Sistemas Periciais: sp_exp1.pl, sp_exp2.pl e base de conhecimento (veiculos1.txt, veiculos2.txt). Materiais da unidade curricular, Mestrado em Engenharia de Inteligência Artificial, ISEP, 2026.

[7] Equipa do projeto. Código fonte do projeto *Intelligent Driving Decision Support System*: ficheiros engine.pl, driving_state.pl, driving_rules.pl, server.pl e classes do pacote com.one2drive.projeto1.prolog, 2026. Consultado em 23 de outubro de 2026.

[8] PPROGIA. Introdução à implementação de Sistemas Periciais em Prolog. Diapositivos da aula 6 (Lecture6_26_27.pdf), Mestrado em Engenharia de Inteligência Artificial, ISEP, 2026.

[9] PPROGIA. Motor de inferência: alguns conceitos. Diapositivos de apoio (ajuda_2026_v10_2.pdf), Mestrado em Engenharia de Inteligência Artificial, ISEP, 2026.

[10] SWI-Prolog. Supporting JSON. https://www.swi-prolog.org/pldoc/man?section=jsonsupport. Consultado em 17 de outubro de 2026.

[11] SWI-Prolog. Declaring predicate properties. https://www.swi-prolog.org/pldoc/man?section=declare. Consultado em 18 de outubro de 2026.

[12] SWI-Prolog. Managing dynamic predicates. https://www.swi-prolog.org/pldoc/man?section=dynpreds. Consultado em 19 de outubro de 2026.

[13] PPROGIA. Perguntas ao Utilizador: que perguntas pode um sistema pericial fazer ao utilizador? Diapositivos (MI_Perguntas_ACM.pdf), Mestrado em Engenharia de Inteligência Artificial, ISEP, 2026.

A | Mapeamento entre código e relatório

Tabela A.1: Localização no código de cada tema do relatório

Tema	Localização principal
Motor de inferência com suporte para explicações	`src/main/prolog/engine.pl`
Estado temporal	`src/main/prolog/driving_state.pl`
Implementação da base de conhecimento para apoio à condução autónoma	`src/main/prolog/driving_rules.pl`
Interface JSON com o resto do sistema	`src/main/prolog/server.pl`
Integração Java	`src/main/java/com/one2drive/projeto1/prolog`
Contrato dos motores	`src/main/java/com/one2drive/projeto1/inference/ExpertSystemBackend.java`
Arbitragem de decisões	`src/main/java/com/one2drive/projeto1/decision/DecisionContext.java`
Testes Prolog	`src/test/java/com/one2drive/projeto1/prolog`
Interface gráfica Web	`frontend/src/app`